# CompFunGene Session 3 - Instructions

## 1. Why do we want to use a coding agent?

- The difference between using AI in a browser chat and as a coding agent is the context and information it has access to.
- If the model LLM is connected to your coding environment, it gains access to your local files, the project structure, and the code you have already written.
- This allows it to provide much more relevant and context-aware suggestions. It can access data, functions, and variables defined in your project, which allows it to generate code that is more likely to work correctly and efficiently.
- It can also help you navigate your codebase, find bugs, and suggest improvements based on the existing code.

## 2. Setting up your coding environment

First, we need to connect Positron to a coding agent. This is done in the `Providers.json` file. Once you have connected to a coding agent, you can use it in any Positron session.

### Steps required to connect Positron to Github Copilot:

- 2.1 Open the Command Palette with `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (macOS). Enter `Posit Assistant: Configure AI Providers`, open `Providers.json` and add:

```
"copilot": {
  "enabled": true
}
```

Your `Providers.json` file should now look something like this. The openai entry is not required:

```
{
  "$schema": "https://assistant.posit.co/schemas/providers.schema.json",
  "version": 1,
  "providers": {
    "openai": { # This is the OpenAI provider, which you can ignore for now
      "baseUrl": "https://api.openai.com/v1"
    },
     "copilot": { # This is the Github Copilot provider, which includes models
like Copilot and Copilot Chat.
      "enabled": true
    }
  }
}
```

- 2.2 Now open Positron's Command Palette (`Ctrl+Shift+P`) and select `Preferences: Open User Settings (JSON)`, click on `open Settings (JSON)`. Add this setting: `"chat.mcp.autostart": "never"`

Your `Settings.json` file should look something like this:

```
{
    "[r]": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "Posit.air-vscode"
    },

    "files.associations": {
        "renv.lock": "json"
    },
    "workbench.colorTheme": "Default Positron Dark",
    "window.autoDetectColorScheme": true,
    "workbench.keybindings.rstudioKeybindings": true,
    "authentication.openai-api.baseUrl": "https://api.openai.com/v1",
    "editor.wordBasedSuggestions": "off",
    "editor.suggest.matchOnWordStartOnly": true,
    "editor.suggest.showModules": false,
    "editor.suggest.showSnippets": false,
    "python.languageServer": "None",
    "telemetry.telemetryLevel": "all",
    "git.openRepositoryInParentFolders": "never",
    "packages.warnMissingInEditor": false,
    "ai.mcp.enabled": true,
    "packages.confirmMissingOnRun": false,
    "chat.disableAIFeatures": false,
    "chat.mcp.autostart": "never"
}
```

- 2.3 Login to your Github Account: In your Positron session, go to `Accounts` (bottom left) -> `Configure LLM providers` -> `Github Copilot` and follow the steps to login to your account.
- 2.4 Restart your Positron session for the changes to apply.

## 3. A few words on model selection and token usage:

- Interacting with an LLM consumes tokens. Using more competent models consumes more tokens for the same task.
- LLM providers sell tokens as part of subscription models, where you get a limited amount of tokens that refills every few hours/days/weeks.
- You can view your token usage in your Positron session, or in your Copilot settings.

- If you start coding using AI on a regular basis, you likely need a paid subscription (20-25€ per month). **Be careful if you are asked to enter an API key during the login process of your coding environment. It could lead to an unlimited payment model, which offers more features but can cost a lot of money.** What you usually want is a non-API key based login, which is usually done via your account on the provider's website.
- When deciding for a model, Token usage / cost per task is a significant factor. High-tier models are usually the most capable, but expensive to use. For most tasks, mid-tier models are sufficient and consume far less tokens.
- The AI market is quite dynamic. Currently, the most popular LLM providers used for coding are OpenAI (ChatGPT Codex), Anthropic (Claude Code) as well as as well as Google (Gemini). A list of the currently leading models and providers can be found here: https://artificialanalysis.ai/leaderboards/models. Notice that the leading models have substantially higher costs per task compared to mid-tier models?

As an example, ChatGPT currently offers the following models for coding tasks (https://learn.chatgpt.com/docs/models?surface=app): Astra > Sol > Terra > Luna. For everyday tasks, I use Terra, for complex interpretations of data I use Sol.

# 4. Prompting: Communicating a programming task

## How to write good prompts for AI-assisted coding

- Imagine the coding agent like another human that you want to assign a task to.
- The programmer can write high quality code, but it has no idea what you want.
- Describe the problem, the context, and what you expect
- The more specific, precise and clear the task is formulated, the more useful the result will be.
- What exactly should the AI do? Specify one clear task.
- What constraints, requirements, or data characteristics should it consider?
- What should the output look like? Code, an explanation, a plan, or a review?
- How will you know the solution is correct? What should be checked?

# 5. Working with an Agents file

Some instructions, for example defining coding styles, preferred coding languages, etc, shouldn't be defined in every prompt. They can be given in an `AGENTS.md` file. You can find a template AGENTS file here: https://github.com/thorohde/CompFunGene_session_3/blob/main/AGENTS.md.

```
## General

- If my instructions are conflicting, unclear or require further decisions or specification, briefly describe the options with their advantages and disadvantages.
```

```
## Coding conventions

- Use R for all analysis code.
- Use descriptive variable and function names.
- Prefer simple, readable solutions.
- Use relative paths, not absolute paths.
- Stick to my style of coding, or the code style already present in the project.
- Keep code compact, don't add input validation routines if not necessary.
- Make one small logical change at a time. Explain what you intend to change and
why. Name the files you intend to modify.
-  Never stage, commit, reset, revert, or discard changes unless explicitly
requested.

## Code review

- Do not assume existing code is correct.
- Look for logical errors, not just syntax errors.
- Identify hidden assumptions and edge cases.
- Check whether joins or filters unexpectedly change the number of observations.
- Prioritize findings by severity.
- Explain how to reproduce each problem.
- Do not edit code unless I request a fix.

## Statistical analysis

- Distinguish exploratory from confirmatory analysis.
- Do not infer causation from correlation.
- State assumptions behind statistical models.
- Check sample sizes and missing observations.
- Distinguish statistical significance from practical importance.
- Never invent numerical results.
```

This `AGENTS.md` file should be placed in the root of your project directory. It will be automatically detected by Positron and used to guide the coding agent's behavior.

AGENTS instructions should be **precise**, **short** and **non-contradicting**. Start small and add instructions when you observe recurring problems. Instructions do not guarantee compliance. Also feel free to ask the AI to generate an AGENTS file for you.

## Coding task

With the AI agent set up, we are ready to start coding. Please open the last weeks file `2_compPertOmics.Rmd` and try to solve task Q2.11 using your coding agent. Try to solve the task in small steps, and ask for help when you get stuck. Also try to apply the prompting strategies

described above. Feel free to go back to previous tasks of the exercises and explore using AI to solve them.

## Optional: Generate a Shiny app to create an interactive volcano plot.

Now, we want to use our AI agent to build an interactive visualization of the data from scratch.

1. At the end of your `2_compPertOmics.Rmd` file, execute the code `saveRDS(geneStats, "geneStats.rds")` to save the previously computed gene statistics data. This generated object will be used by the newly generated Shiny app.
2. Open the `shiny_app.qmd` file.
3. Tell the AI agent to build an R shiny app which visualizes the data from the newly generated geneStats.rds in a volcano plot. You may use the instructions below, feel free to experiment and play around. You might also try and create full scientific reports based on the data.

```
Build an R shiny app which visualizes the data from geneStats.rds in a volcano plot. Offer the option to filter the data based on p-value and log fold change thresholds. The app should allow users to adjust these thresholds using sliders and display the resulting volcano plot interactively. Place the sliders below the plot.
```

## Takeaways: If AI can write all the code we need, what is left to learn?

Being able to code is not the skill it was five years ago. **Problem formulation, code comprehension, debugging, and validation remain essential.**

1. Understand the problem before generating code.
2. Ask AI for small, understandable changes. Ask it to slow down and explain.
3. Don't accept code you cannot explain at the relevant level.
4. Test results, not just whether the code runs.
5. Use AI to accelerate learning, not bypass it.